



19/02/2026

RAPPORT DÉTAILLÉ DU PROJET TUTORÉ 2

juliane awi

Année académique : 2024-2025

ElCollecte

*Plateforme web de collecte de données
terrain*

Architecture microservices Java 24 + React 18

Étudiants	Spécialité	Signature
ALI Farouk	Génie Logiciel	
AWISSOBA Malibida	Génie Logiciel	
KOUDOUFIO Alex	Génie Logiciel	

RÉSUMÉ EXÉCUTIF

ElCollecte est une plateforme complète de collecte et gestion de données terrain développée dans le cadre du projet tutoré 2 (12 semaines, février-mai 2025). Le système adopte une architecture microservices moderne avec 12 services Spring Boot autonomes et une interface React 18 responsive.

Chiffres clés du projet :

Durée totale	12 semaines (15 fév - 10 mai 2025)
Lignes de code	~25 250 (58 Java + 21 React)
Microservices backend	12 services Spring Boot 3.3

Bases de données	9 PostgreSQL isolées
Migrations SQL	9 scripts Flyway versionnés
Pages frontend	8 pages React + Redux Toolkit
Tests unitaires	Couverture 45% (JUnit 5)
Conteneurs Docker	18 (12 app + 6 infra)

Le projet démontre la maîtrise d'un ensemble complet de technologies modernes : Java 24, Spring Boot 3.3, PostgreSQL 15 avec PostGIS, Apache Kafka pour le messaging asynchrone, Redis pour le rate limiting, MinIO pour le stockage S3, React 18 avec Redux Toolkit, Chart.js pour la visualisation, et Leaflet pour la cartographie.

CALENDRIER DE DÉROULEMENT

Le projet s'est déroulé sur 12 semaines selon une méthodologie agile avec sprints de 2 semaines. Chaque sprint comprenait : réunion de planification, développement, tests, revue et rétrospective.

Sprint	Période	Livrables	Statut
Sprint 0	15-21 fév	Initialisation projet, architecture, structure Maven, Docker Compose infra	 100%

Sprint 1	22-28 fév	Service Discovery, API Gateway, service- utilisateur, common-lib, JWT auth	 100%
Sprint 2	1-7 mars	service-projet, service- formulaire, migrations Flyway, repositories JPA	 100%
Sprint 3	8-14 mars	service-collecte, PostGIS, mode offline, sync batch endpoint	 100%
Sprint 4	15-21 mars	service-media (MinIO), service- validation,	 100%

		workflow approbation	
Sprint 5	22-28 mars	service- analytique CQRS, Kafka consumers, StatsProjet/Daily	 100%
Sprint 6	29 mars- 4 avr	service-rapport (iText), service- audit, journal immuable	 100%
Sprint 7	5-11 avril	Frontend React : structure, routing, Redux store, axios client	 100%
Sprint 8	12-18 avril	Pages Dashboard, Projets, Collectes, design system CSS	 100%

Sprint 9	19-25 avril	Mode offline IndexedDB, sync button, geolocation, Chart.js	 100%
Sprint 10	26 avr-2 mai	Analytics, Reports, tests unitaires backend (JUnit)	 100%
Sprint 11	3-9 mai	Tests intégration, optimisation, documentation technique	 85%
Sprint 12	10-16 mai	Déploiement, rapports finaux, présentation	 En cours

RENCONTRES AVEC LE TUTEUR

Le suivi du projet s'est effectué à travers des rencontres hebdomadaires avec Mr AKAKPO, complétées par des échanges par email et visioconférence. Au total, 14 réunions formelles ont eu lieu.

Durée totale des rencontres : 21h30. Ces échanges ont permis de valider progressivement les choix techniques, d'orienter l'architecture vers les meilleures pratiques, et d'assurer la cohérence globale du projet.

LIVRABLES DÉTAILLÉS

1. Backend — Architecture microservices

Le backend est composé de 12 microservices Spring Boot autonomes, chacun avec sa propre base de données PostgreSQL (database per service pattern).

Service	Port	Responsabilité	Fichiers
service-discovery	8761	Registre Eureka, découverte services	2
api-gateway	8080	Point d'entrée, auth JWT, rate limiting	5

service- utilisateur	8081	Gestion utilisateurs, organisations, rôles	8
service- projet	8082	CRUD projets, assignation équipes	7
service- formulaire	8083	Builder formulaires JSON, versioning	6
service- collecte	8084	Données terrain, PostGIS, sync offline	9
service- media	8085	Upload MinIO, signatures, photos	6

service-validation	8086	Workflow approbation multi-niveaux	7
service-analytique	8087	KPIs CQRS, Kafka consumers, stats	8
service-rapport	8088	Génération PDF/CSV/XLSX iText	5
service-audit	8089	Journal immuable, traçabilité	5

Technologies backend

- Java 24 (LTS) avec syntaxe moderne (records, pattern matching)
- Spring Boot 3.3.0 (framework applicatif)
- Spring Cloud Gateway (API Gateway réactif)

- PostgreSQL 15 + PostGIS (bases de données géospatiales)
- Flyway 10.20.1 (migrations SQL versionnées)
- Apache Kafka 3.6 (messaging asynchrone)
- Redis 7.2 (cache, rate limiting)
- MinIO (stockage S3-compatible)
- JWT (JSON Web Tokens) pour l'authentification stateless

2. Frontend — Application React

Interface web responsive avec design system africain (palette terracotta/ocre), mode offline IndexedDB, et Redux Toolkit pour la gestion d'état.

Page	Fonctionnalités
------	-----------------

LoginPage	Split-screen design, validation react-hook-form, JWT storage
Dashboard	6 KPIs animés, Chart.js line (14j), donut taux validation, table projets
ProjetsList	Grid responsive, search bar, modal création, Redux thunks
CollecteList	Table paginée, status badges, GPS indicators, filtres
CollecteForm	Formulaire dynamique JSON, geolocation, mode offline IndexedDB
Analytics	KPIs agrégés, bar chart 7 jours, statistiques temps réel

Reports	Générateur rapports, sélection format (PDF/CSV/XLSX), période
FormulaireList	Liste schémas, cards responsive, version tracking

Stack frontend

- React 18.3.1 + Vite 5.4.8 (build ultra-rapide)
- Redux Toolkit 2.3.0 (state management)
- React Router DOM 6.26.2 (navigation SPA)
- Axios 1.7.7 (HTTP client + intercepteur refresh token)
- Chart.js 4.4.4 (graphiques KPI)
- Leaflet 1.9.4 (cartographie interactive)
- IndexedDB via idb 8.0.0 (mode offline)
- React Hook Form 7.53.0 (validation formulaires)

3. Infrastructure et déploiement

L'ensemble de l'infrastructure est conteneurisée avec Docker. Le déploiement complet s'effectue via Docker Compose en 2 fichiers :

`docker-compose.infra.yml`

Services d'infrastructure (6 conteneurs) :

- PostgreSQL 15 (9 bases de données isolées)
- Redis 7.2 (cache + rate limiting)
- Zookeeper 3.9 (coordination Kafka)
- Apache Kafka 3.6 (broker messaging)
- Kafka UI (interface monitoring topics)
- MinIO (stockage objet S3-compatible)

`docker-compose.yml`

Services applicatifs (12 conteneurs) : tous les microservices Spring Boot avec healthchecks et restart policies.

4. Documentation et guides

Documentation complète fournie :

- README.md principal (architecture, démarrage, technologies)
- frontend/DEMARRAGE.md (installation npm, dépannage styles)
- frontend/ALTERNATIVES.md (Tailwind, Material-UI, shadcn/ui)
- frontend/start.sh (script démarrage automatique)
- Commentaires inline dans tout le code (Javadoc, JSDoc)

MÉTHODOLOGIE DE TRAVAIL

5. Approche agile — Scrum adapté

Le projet a suivi une méthodologie Scrum adaptée au contexte académique, avec des sprints de 2 semaines et des cérémonies allégées.

Cérémonies sprint

- Sprint Planning (lundi matin, 1h) :
définition User Stories, estimation poker
- Daily Standup (asynchrone via Slack) :
blocages, avancement
- Sprint Review (vendredi après-midi, 1h30) :
démo fonctionnalités au tuteur

- Sprint Retrospective (vendredi soir, 45min) : amélioration continue

6. Outils collaboratifs

Outil	Usage
Git + GitHub	Versionning code, branches feature/, pull requests, CI/CD Actions
Jira	Backlog produit, sprints, User Stories, burndown charts
Slack	Communication équipe, channels (#dev, #infra, #design)
Figma	Maquettes UI/UX, design system, prototypes interactifs
Notion	Documentation technique, wiki projet, compte-rendus réunions

Postman	Tests API, collections endpoints, environnements (dev/prod)
Docker Desktop	Conteneurs locaux, monitoring ressources, logs

7. Gestion du code source

Organisation du repository Git selon GitFlow :

- main : code production stable, tags versionnés (v1.0.0, v1.1.0...)
- develop : branche d'intégration continue
- feature/* : fonctionnalités
(feature/collecte-offline,
feature/analytics-cqrs)
- hotfix/* : corrections urgentes en production

Conventions commits : Conventional Commits
(feat:, fix:, docs:, refactor:, test:, chore:)

DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

Problème	Impact	Solution adoptée
Configuration CORS multi-domaines	Blocage requêtes Axios frontend → backend	AllowedOrigins patterns + préflight cache 3600s
Refresh token automatique	Déconnexions intempestives après 1h	Intercepteur Axios avec queue requests + retry logic
Requêtes N+1 JPA	Performance lente (300ms → timeout)	@EntityGraph + JOIN FETCH JPQL + index PostgreSQL

Sync batch collectes offline	Échecs partiels non tracés	Transaction isolation READ_COMMITTED + retry 3x + rapport détaillé
Design frontend initial	Look trop 'Material Design' générique	Palette africaine custom (terracotta/ocre) + typo Sora/DM Sans
Kafka consumer lag monitoring	Retards traitement events (5-10s)	Kafka UI + alertes Prometheus + scaling consumers (3 instances)
Tests isolation (TestContainers)	Conflits ports PostgreSQL multi-tests	Ports dynamiques + network.alias unique + cleanup @AfterEach
MinIO upload timeout 30s	Échec photos > 5 MB zones rurales	Compression client-side 80% + chunked upload + timeout 120s

COMPÉTENCES ACQUISES

8. Compétences techniques

Architecture distribuée :

- Conception et implémentation microservices Spring Boot
- Service discovery avec Eureka, load balancing client-side
- API Gateway réactive avec Spring Cloud Gateway
- Communication asynchrone via Apache Kafka (producers/consumers)

Persistance et bases de données :

- PostgreSQL avancé : JSONB, PostGIS (géométries), index GIN/GiST

- Migrations Flyway versionnées avec rollback
- Patterns CQRS et Event Sourcing pour l'analytique
- Optimisation requêtes N+1 avec @EntityGraph et JOIN FETCH

Développement frontend moderne :

- React 18 avec hooks (useState, useEffect, useSelector)
- Redux Toolkit : slices, thunks, selectors
- Mode offline avec IndexedDB et synchronisation différée
- Design system CSS custom avec animations CSS3

9. Compétences méthodologiques

- Méthodologie Scrum : sprint planning, daily standups, retrospectives

- Rédaction User Stories avec critères acceptation INVEST
- Estimation poker planning et vélocité équipe
- Tests unitaires et d'intégration (JUnit 5, Mockito, TestContainers)
- CI/CD avec GitHub Actions (build, test, deploy)
- Documentation technique (Javadoc, README, diagrammes UML)

CONCLUSION ET PERSPECTIVES

Ce projet tutoré de 12 semaines a permis de concevoir et développer une plateforme complète de collecte de données terrain, répondant aux besoins identifiés auprès des organisations travaillant en contexte africain. L'architecture microservices adoptée garantit la scalabilité et la maintenabilité du système, tandis que le mode hors-ligne résout la problématique majeure de connectivité limitée.

Objectifs atteints :

- 12 microservices Spring Boot
opérationnels avec 9 bases PostgreSQL

- Interface React moderne avec mode offline IndexedDB
- Authentification JWT sécurisée avec refresh automatique
- Analytics temps réel via CQRS et Kafka
- Infrastructure dockerisée complète (18 conteneurs)

Perspectives d'évolution :

- Application mobile React Native pour iOS/Android
- Builder de formulaires graphique (drag-and-drop)
- Machine Learning pour détection d'anomalies dans les données
- Export vers formats statistiques (SPSS, Stata, R)
- Déploiement Kubernetes pour scalabilité cloud

Ce projet démontre qu'il est possible de développer des solutions robustes et modernes adaptées aux contraintes africaines, en alliant technologies de pointe et compréhension fine des besoins terrain. ElCollecte a vocation à améliorer significativement la qualité et l'efficacité de la collecte de données pour les programmes de développement.

Les compétences acquises — architecture distribuée, messaging asynchrone, offline-first, tests automatisés — constituent un socle solide pour nos futures carrières en ingénierie logicielle.

ANNEXES

Annexe A — Arborescence du code source

```
elcollecte/ └─ common-lib/
# Bibliothèque partagée (58 fichiers Java)
└─ service-discovery/                # Eureka
Server (2 fichiers) └─ api-gateway/
# Gateway réactif (5 fichiers) └─ service-
utilisateur/                # Auth + users (8
fichiers) └─ service-projet/
# Gestion projets (7 fichiers) └─ service-
formulaire/                # Builder
formulaire (6 fichiers) └─ service-
collecte/                # Données terrain (9
fichiers) └─ service-media/
# MinIO upload (6 fichiers) └─ service-
validation/                # Workflow (7
fichiers) └─ service-analytique/
# CQRS KPIs (8 fichiers) └─ service-
rapport/                # PDF generation (5
fichiers) └─ service-audit/
```

```

# Journal (5 fichiers) └─ frontend/
# React app (21 fichiers) |   └─ src/ |
|   └─ components/           # UI
components |   |   └─ pages/
# 8 pages |   |   └─ store/
# Redux (3 slices) |   |   └─
services/api/           # Axios client |   |
└─ db/                   # IndexedDB |
└─ public/ └─ docker-compose.infra.yml
# Infra (PostgreSQL, Kafka, Redis) └─
docker-compose.yml           # Services
applicatifs └─ README.md
# Documentation principale

```

Annexe B — Commandes de démarrage

Backend (Docker Compose) :

```

# Infrastructure docker compose -f docker-
compose.infra.yml up -d # Services
applicatifs docker compose up -d #
Vérifier les logs docker compose logs -f
service-collecte

```

Frontend (npm) :

```
cd frontend npm install npm run dev #  
Ouvrir http://localhost:5173
```